

OpenGotchi Jobs on Arc

ETHGlobal ETHOnline 2026 · Arc track · a voice-driven pet that hires and pays AI agents on Arc · repo github.com/Pranav9931/opengotchi-ethonline · written 13 Sept 2026

This repository is an extension of our flagship gotchiOS. gotchiOS is the ESP32-S3 pet operating system (wake word, voice rail, MQTT, MicroPython app runtime, AMOLED UI). Anyone can flash it onto their own ESP32 device from the open-source releases at <https://github.com/opengotchi/gotchiOS-releases>. Everything in this repository was written during the hackathon (4–13 Sept 2026) and runs *on top of* the stock firmware: no firmware code is modified or included.

1. What it does, in one sentence

Say "Jarvis, what's the weather in Berlin?" to the pet: the pet agent opens an **ERC-8183 job on Arc**, funds **USDC escrow**, a worker agent with an **ERC-8004 onchain identity** does the work and submits the deliverable hash, the pet verifies the hash, **settles the USDC on Arc**, records feedback on the worker's reputation, then speaks the answer and shows the receipt on its screen.

2. Was a contract deployed? Addresses

No contract of our own was deployed. The whole flow runs on the contracts Arc documents for its agentic economy, already live on Arc testnet (chain id 5042002, USDC is the native gas token). We are a client of those contracts, which is the point: the standards are the product.

Contract	Address (Arc testnet)	Used for
USDC (native gas, ERC-20 interface, 6 dp)	0x3600000000000000000000000000000000000000	approve + escrow + settlement
ERC-8183 AgenticCommerce (reference)	0x0747EEf0706327138c69792bF28Cd525089e4583	createJob · setBudget · fund · submit · complete · getJob
ERC-8004 IdentityRegistry	0x8004A818BFB912233c491871b3d84c89A494BD9e	worker registers itself (NFT identity)
ERC-8004 ReputationRegistry	0x8004B663056A597Dffe9eCcC1965A193B7388713	pet gives feedback after each job
ERC-8004 ValidationRegistry	0x8004Cb1BF31DAf7788923b405b754f57acEB4272	ABI bundled, not exercised in the demo

Wallets (self-managed viem accounts, keys only in the gitignored `.env`)

Role	Address	Balance at writing
Pet agent — client + evaluator, funds escrow	0xfE8C4A56C628eb89c7F01A0671850DCDB9C67AD8	16.50 USDC
Worker agent — provider, ERC-8004 agentId 894780	0x99D60AAeD7e747D26B20FEd34A93ECC06f1C0372	3.17 USDC

Worker identity registration tx: 0x1ded823a52981e79e971a8bff08e34bca727900a3cc1dcf91561ea0b5a4523a4. Funding came from the Circle faucet (20 USDC to the pet), and the pet forwarded 3 USDC to the worker onchain (`bun run wallet fund-worker 3`).

Onchain activity produced during the hackathon

- **17 jobs settled** on Arc testnet (ids 186256 – 186274), 4 declined by policy, 2 failed and reported. **0.230 USDC** paid to the worker in escrow settlements.
- Every settled job = **7 Arc transactions**: createJob, setBudget, approve, fund, submit, complete, giveFeedback → about 120 transactions plus the identity mint. Typical wall-clock per job: 10–18 s including the worker's public-data fetch.

- Example, job 186274 ("Watch the weather in Berlin."): complete tx `0xdc74096aad609d7653ecaeb25b50078e005d7fa1696b073218035c51880996ee`, feedback tx `0xe661c19ba744e6d49b72e7d5d131b195ff1d66b786ce4ff84865ca5af7333af0`. Explorer: <https://testnet.arcscan.app>.

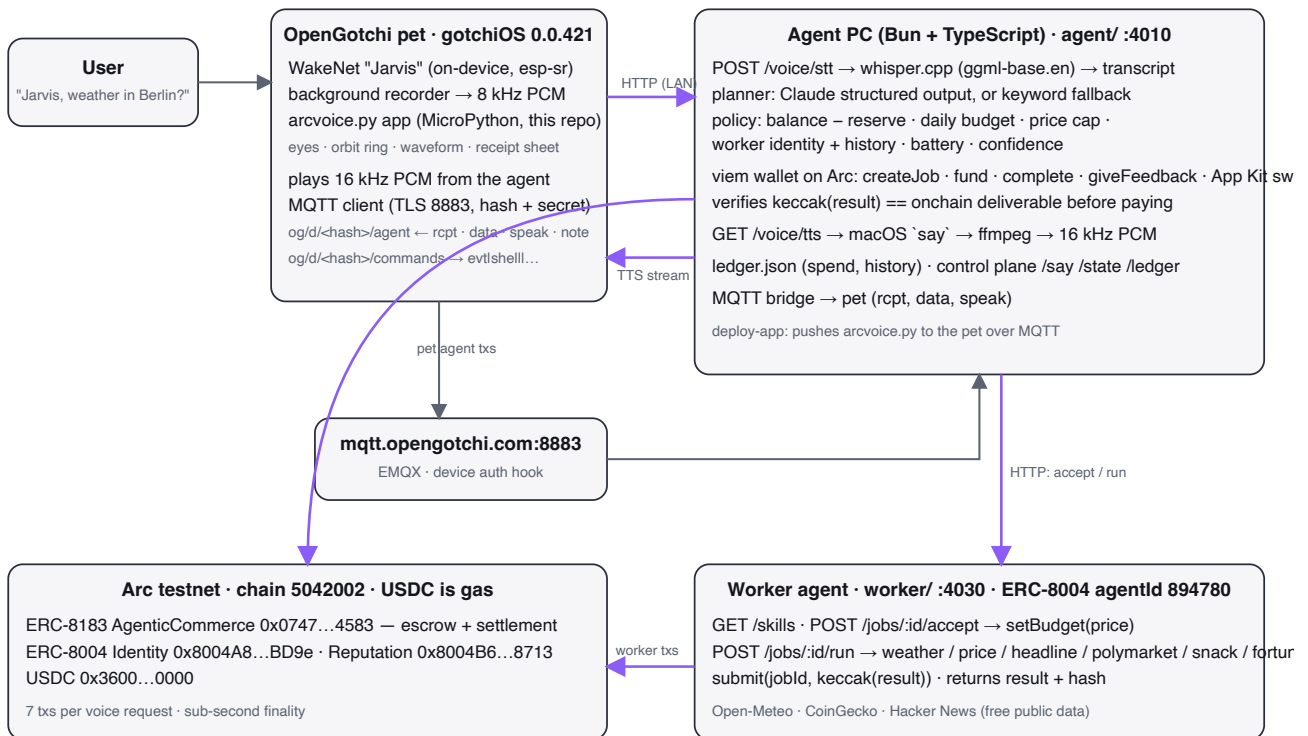
Buying tokens by voice (Circle App Kit)

"Jarvis, buy one dollar of bitcoin" is a **real same-chain swap on Arc testnet** through Circle App Kit (`@circle-fin/app-kit` with the viem private-key adapter): `kit.estimateSwap` for the quote, then `kit.swap({ from: { chain: "Arc_Testnet" }, tokenIn: "USDC", tokenOut, amountIn })`. Arc testnet lists USDC, EURC and cirBTC, so the pet can buy **bitcoin (cirBTC)** or **euros (EURC)**; anything else ("buy ethereum") is declined by name because it does not exist on Arc. Purchases are capped by `MAX_SWAP_USD` (2) and the reserve rule. Proof: 0.5 USDC → 0.399934 EURC `0xe76f13fb1c6a7b50e0e961268f992e1c80ffc8f963ea1f8847e298be21b72747`; 1 USDC → 0.00000271 cirBTC `0xc91f8e3ea2c1bffb978281585401d62a97a8f2b77ca2f8d06298049f5968a0b`.

Prediction markets

"What are the odds of a Fed rate cut?" is a worker job (skill `polymarket`, 0.02 USDC) that queries Polymarket's public Gamma API, picks the highest-volume active market for the topic and reads out the Yes/No prices. Job 186283 answered: "Will there be no change in Fed interest rates after the September 2026 meeting? Yes 22 percent, No 79 percent." The complete list of prompts that were run on hardware is in `docs/questionnaire.md`.

3. The pipeline



Sequence of one request

1. Wake word fires on the pet (or tap). `arcvoice.py` records via the firmware's background recorder with a live waveform and stops on silence.
2. PCM is POSTed over the LAN to the agent; `whisper.cpp` transcribes in ~0.3 s.
3. The planner maps the transcript to a worker skill and parameters with a confidence score.
4. The policy engine approves or declines from live signals; declines are spoken and shown.
5. Pet: `createJob(provider=worker, evaluator=pet)`. Worker: `setBudget`. Pet: `approve` + `fund` (escrow). Worker: does the work, `submit(keccak(result))`. Pet: checks the hash, `complete` → USDC to worker. Pet: `giveFeedback`.
6. Receipt (status, skill, job, price, tx, balance, today, result) is sent to the pet as a single MQTT message, then the answer is spoken through the pet's speaker via streamed TTS.

4. The device app: `device/arcvoice.py`

A single-file MicroPython app (~18 KB) that runs inside the stock gotchiOS app runtime. It is **deployed over MQTT**, not flashed: `bun run deploy-app` sends the file to `og/d/<hash>/app/deploy` with `launch:true`; the firmware stores it in `/littlefs/apps/`, so it also appears in the launcher for manual start. Before deploying, the script asks a running instance to `exit` cleanly (deploying over a live instance rebooted the pet twice during development).

Aspect	Configuration
Agent endpoint	<code>__AGENT_URL__</code> is substituted at deploy time with <code>http://<PC LAN IP>:4010</code> (override with <code>AGENT_PUBLIC_URL</code> or a CLI argument). The pet and the PC must share a network.
Wake word	<code>audio.wake_ready()/wake_start()/wake_detected()</code> : the firmware's on-device WakeNet "Jarvis" model. Armed at idle, disarmed while recording or speaking (the detector and recorder cannot share the mic), re-armed after. Tap is the fallback trigger.
Recording	<code>audio.rec_start(16000, 12, False)</code> background recorder; the firmware delivers 8 kHz PCM16 into a ring. Energy VAD: noise floor learned in the first 300 ms, threshold = 3× floor + 350, stop after 800 ms of silence once speech was heard, minimum 1.2 s, maximum 7 s, give up after 4 s of nothing. Sent as <code>POST /voice/stt</code> with <code>X-Sample-Rate: 8000</code> .
Playback	<code>http.stream(GET /voice/tts?t=...)</code> feeds 16 kHz PCM16 chunks into <code>audio.play()</code> ; odd trailing bytes are carried to the next chunk so samples never misalign.
MQTT directives (from agent)	<code>rcpt k us v rs k us v ...</code> whole receipt in one message (the firmware inbox holds 8; status is applied last) · <code>data k v</code> single field / <code>step</code> · <code>speak text</code> · <code>note text</code> <code>toast</code> · <code>ping</code> → <code>evt shell pong:arcvoice</code> · <code>exit</code> .
Events (to agent)	<code>evt shell ready:arcvoice</code> , <code>wake:armed</code> , <code>rcpt:<status></code> , <code>error:<repr></code> on crash.
UI	gotchiOS token sheet: dark AMOLED, violet gradient, Space Grotesk / Plex faces via <code>display.ftext</code> , rounded surfaces. Milady eye sprites react to state (spiral transcribing, heart settled, sleepy held, teary failed). Six-node orbit ring lights one node per Arc transaction with a comet while working; particle burst on settlement; live waveform while listening; scrolling hint ticker at idle.
Gestures	tap = talk · swipe down / long press / button 1 = exit to launcher.

5. The PC side: agent + worker

Processes

Process	Port	Role
<code>bun run worker</code> (worker/src)	4030	Registers an ERC-8004 identity on first start (stored in <code>worker.json</code>), serves the skill catalogue, sets budgets, does jobs, submits deliverable hashes.
<code>bun run agent</code> (agent/src)	4010	MQTT bridge to the pet, STT/TTS endpoints, planner, policy, Arc job lifecycle, ledger, control plane.

Speech

- Speech-to-text:** `whisper.cpp` (`whisper-cli`, Homebrew package `whisper-cpp`) with the `ggml-base.en.bin` model (148 MB, from huggingface.co/ggerganov/whisper.cpp), run locally with `-nt -l en`. About 200–350 ms per utterance on Apple Silicon. `ffmpeg` resamples the pet's 8 kHz audio to 16 kHz first.
- Text-to-speech:** macOS built-in `say` (system voice, override with `SAY_VOICE`) rendered at 16 kHz and converted by `ffmpeg` to raw signed 16-bit mono PCM, cached per sentence. On Linux, replace `synthesize()` in `agent/src/voice.ts` with `piper` or `espeak-ng` producing the same raw format.
- Planner:** Claude (`claude-opus-5`, structured outputs) when `ANTHROPIC_API_KEY` is set; a keyword planner otherwise. The hosted OpenGotchi voice API is not used at all.

Running on a completely new PC

```
# 1. tools (macOS; Linux: same minus `say`, see Speech above)
brew install node bun ffmpeg whisper-cpp
git clone https://github.com/Pranav9931/opengotchi-ethonline && cd opengotchi-ethonline
bun install
mkdir -p models && curl -L -o models/ggml-base.en.bin \
  https://huggingface.co/ggerganov/whisper.cpp/resolve/main/ggml-base.en.bin

# 2. configuration
cp .env.example .env
bun run wallet new          # prints AGENT_PRIVATE_KEY and WORKER_PRIVATE_KEY → paste into .env
# fund the agent address with Arc testnet USDC at https://faucet.circle.com (20 USDC is plenty)
bun run wallet fund-worker 3 # pet sends 3 USDC to the worker for gas
# .env: DEVICE_HASH=<pet hash> GOTCHI_DEVICE_SECRET=<pet's provisioned secret>
#       WHISPER_MODEL=./models/ggml-base.en.bin (ANTHROPIC_API_KEY optional)

# 3. run (two terminals), then push the app to the pet
bun run worker
bun run agent
bun run deploy-app          # substitutes this PC's LAN IP, exits any running copy, deploys +
                             launches
```

The pet must run gotchiOS $\geq$ 0.0.421 (flash from github.com/opengotchi/gotchiOS-releases), be provisioned (it has a device hash and secret), and be on the same Wi-Fi as the PC. MQTT goes through [mqtt://mqtt.opengotchi.com:8883](https://mqtt.opengotchi.com:8883); the PC logs in with the pet's hash as username and its secret as password, with a `gotchi-bridge-*` client id (the broker rejects `agent-*` ids). Without a device, `curl -X POST localhost:4010/say -d '{"text":"..."}'` drives the identical flow.

Policy defaults (`.env`)

Variable	Default	Rule
MAX_PRICE_USD	0.05	per-job cap; the onchain quote must also not exceed the catalogue price
DAILY_BUDGET_USD	0.50	spent today (ledger.json) + price must fit
MIN_RESERVE_USD	0.20	balance after the job must stay above this
worker history	—	a worker with no completed jobs only gets jobs $\leq$ half the cap; no ERC-8004 identity → never paid
battery (telemetry)	—	below 15 % only snack jobs are allowed
confidence	—	below 0.6 the pet asks you to repeat

6. Repository map

shared/arc.ts	Arc testnet clients (viem `arcTestnet`), ERC-8183 + ERC-8004 + ERC-20 ABIs, helpers	
agent/src/index.ts	the pipeline: voice → plan → policy → job lifecycle → receipt; control plane	
agent/src/voice.ts	/voice/stt (whisper.cpp) and /voice/tts (macOS say → PCM)	
agent/src/device.ts	MQTT bridge: subscribes commands + telemetry, sends rcpt/data/speak/note	
agent/src/brain.ts	Claude planner (zod structured output) + keyword fallback	
agent/src/policy.ts	decision logic on live signals	agent/src/ledger.ts spend + history
agent/src/skills.ts	worker catalogue client	agent/src/config.ts env
agent/src/wallet-cli.ts	new status fund-worker	agent/src/deploy-app.ts push the app to the pet
worker/src/index.ts	ERC-8004 registration, /skills, /jobs/:id/accept, /jobs/:id/run	
worker/src/skills.ts	weather · crypto_price · headline · polymarket (Gamma API) · snack · fortune	
agent/src/swap.ts	"buy" by voice: Circle App Kit swap USDC → cirBTC / EURC on Arc	
device/arcvoice.py	the on-device app (deployed over MQTT)	device/receipt.py legacy fragment for
agent_shell		
docs/	this document, architecture.md (mermaid), submission.md	

7. Honesty notes

- Arc testnet only; Arc mainnet was not yet in Circle's stack during the hackathon. Switching is the viem chain object plus the mainnet registry addresses once published.
- Wake word, mic, speaker and MQTT are gotchiOS features used through their public MicroPython APIs; the closed firmware source is not in this repo.
- Worker skills use free public data. What is bought and sold is the job and its onchain settlement.
- Wallets are self-managed keys for the demo; Arc's tutorials show the same flow with Circle developer-controlled wallets.